# Homework 3

## 1. Basic tensor arithmetic

The OpenCV input is converted from BGR to RGB and stored as a float32 tensor. Ordinary brightening adds 100.0 without clamping, so values may exceed 255. Saturation arithmetic converts through int16, clamps to [0, 255], and returns uint8.

| Result | Shape | dtype | Minimum | Maximum |
|---|---|---|---|---|
| RGB tensor | 321 x 433 x 3 | torch.float32 | 23.0 | 251.0 |
| Bright +100 | 321 x 433 x 3 | torch.float32 | 123.0 | 351.0 |
| Saturated +100 | 321 x 433 x 3 | torch.uint8 | 123 | 255 |

Original RGB image



Float brightening, clipped only for display



Saturation arithmetic



## 2. Gaussian-noise augmentation

Independent Gaussian noise with mean 0 and standard deviation 100 gray levels is added on the 0-255 scale. The result is divided by 255, clamped to [0, 1], and returned as float32. A fixed seed of 596 is used only to reproduce this report figure.

| Result | Shape | dtype | Minimum | Maximum |
|---|---|---|---|---|
| Noisy image | 321 x 433 x 3 | torch.float32 | 0.000 | 1.000 |

Original RGB image



Gaussian-noise augmentation



Clipping creates pixels at exactly 0 and 1 where large negative or positive noise would otherwise leave the valid image range.

## 3. Image normalization

For self-normalization, RGB channel means and sample standard deviations are computed in float64 over height and width. No clamp is applied, preserving zero mean and unit standard deviation. The preview below is min-max mapped only for display; the returned tensor remains standardized.

| Channel | Input mean | Input std | Output mean | Output std |
|---|---|---|---|---|
| Red | 119.2620 | 60.5137 | -1.10e-16 | 1.000000 |
| Green | 139.6622 | 54.6588 | -1.40e-16 | 1.000000 |
| Blue | 123.3738 | 60.5748 | -1.12e-17 | 1.000000 |

ImageNet normalization first scales RGB pixels to [0, 1], then applies (pixel - mean) / std with means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]. Per the assignment, the final ImageNet result is clamped to [0, 1].

Self-normalized tensor, display mapping



ImageNet-normalized and clamped



## 4. Dimension rearranging

The RGB float32 image is rearranged from H x W x C to N x C x H x W. permute(2, 0, 1) produces C x H x W, and unsqueeze(0) adds a batch dimension N=1.

| Stage | Shape | dtype | Minimum | Maximum |
|---|---|---|---|---|
| Input tensor | 321 x 433 x 3 | torch.float32 | 23.0 | 251.0 |
| Rearranged | 1 x 3 x 321 x 433 | torch.float32 | 23.0 | 251.0 |

Pixel content is unchanged by dimension rearrangement



The operation changes tensor indexing but does not resize, recolor, or numerically modify the image. For example, output[0, :, y, x] stores the RGB vector for pixel (y, x).

## 5. Stride-2 Scharr-x convolution

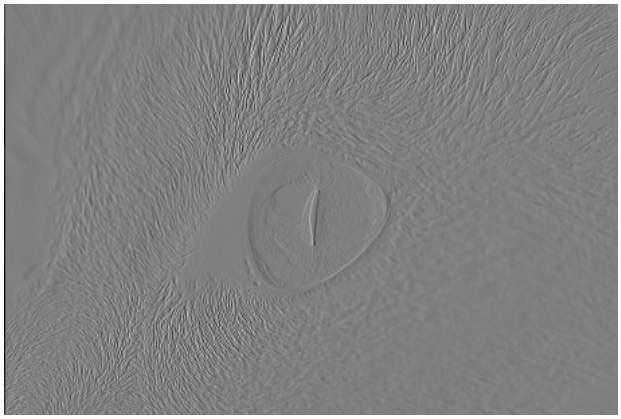
The grayscale cat-eye image is converted to float32 and convolved with the 3 x 3 Scharr-x kernel [[-3,0,3],[-10,0,10],[-3,0,3]]. Zero padding of one pixel preserves size for stride 1; stride 2 samples every second output position.

| Result | Shape | dtype | Minimum | Maximum |
|---|---|---|---|---|
| Input grayscale | 610 x 914 | uint8 | 1 | 230 |
| Stride output | 305 x 457 | torch.float32 | -3485.0 | 2600.0 |

Original grayscale image



Signed Scharr-x response: gray=0



The output is a two-dimensional torch.FloatTensor with shape 305 x 457. The signed response was verified exactly against SciPy convolve2d with constant-zero borders followed by [::2, ::2] sampling.